For the local machine to remote machine file transfer, we use the following:

```
scp local_file_path user@remote_host:destination_file_path
```

For a remote machine to local machine transfer:

```
scp user@remote_host:remote_file_path local_file_path
```

For a remote machine to remote machine transfer:

```
scp user1@remote_host1 user2@remote_host2
```

You can even use wildcards to select files:

```
scp ./home/slynux/*.txt /home/gnubox/scp_example/
```

SFTP stands for Secure File Transfer Protocol. It is a secure implementation of the traditional FTP protocol with SSH as the backend. Let us take a look at how to use the *sftp* command:

```
sftp user@hostname
```

For example:

```
slynux@slynux-laptop:~$ sftp slynux-laptop
Connecting to slynux-laptop...
slynux@slynux-laptop's password:
sftp> cd django
sftp> ls -l
drwxr-xr-x   2 slynux  slynux     4096 Apr 30 17:33 website
sftp> cd website
sftp> ls
__init__.py    __init__.pyc   manage.py      settings.py    settings.pyc
urls.py        urls.pyc       view.py        view.pyc
sftp> get manage.py
Fetching /home/slynux/django/website/manage.py to manage.py
/home/slynux/django/website/manage.py         100%  542     0.5KB/s   00:01
sftp>
```

If the port for the target SSH daemon is different from the default port, we can provide the port number explicitly as an option, i.e., *-oPort=port_number*.

- Some of the commands available under *sftp* are:
- *cd*—to change the current directory on the remote machine
- *lcd*—to change the current directory on localhost
- *ls*—to list the remote directory contents
- *lls*—to list the local directory contents
- *put*—to send/upload files to the remote machine from the current working directory of the localhost
- *get*—to receive/download files from the remote machine to the current working directory of the localhost

sftp also supports wildcards for choosing files based on patterns.

SSH over GUI file managers

In GNOME, we can use the SSH protocol to navigate remote filesystems in the Nautilus file manager. It works as a GUI implementation of *sftp*. Type *ssh://user@hostname[:port]* at the address bar. It will prompt you for the password of the 'user' and then mount the remote filesystem. After that, we can navigate the filesystem just as with locally mounted disk data.

As for KDE users, you can use the *fish* protocol in Dolphin or Konqueror to browse remote filesystems. Type *fish://user@hostname[:port]* in the location bar and press *Enter*. It will again prompt for the remote user's password.

Running XWindow applications remotely

Well, the good news is that SSH is also a good enough protocol that can aid you to run applications other than terminal utilities remotely, with the help of X11 forwarding. To enable X11 forwarding, add the following line in */etc/ssh/ssh_config*, the configuration file.

```
ForwardX11 yes
```

Now to launch the GUI apps remotely, execute *ssh* commands with the *-X* option. For example:

```
ssh -X slynux-laptop 'vlc'
```

Port forwarding

One of the significant uses of SSH is port forwarding. SSH allows you to forward ports from client to server and server to client. There are two types of port forwarding: local and remote. In local port forwarding, ports from the client are forwarded to server ports. Thus the locally forwarded port will act as the proxy port for the port on the remote machine.

To establish local port forwarding, use the following code:

```
ssh -L local_port:remote_host:remote_port
```

For example:

```
ssh -L 2020:slynux.org:22
```

Here, it forwards local port 2020 to slynux.org's ssh port 22. Thus, we can use:

```
ssh localhost -p 2020
```

...instead of:

```
ssh slynux.org
```

In remote port forwarding, ports from the server are forwarded to a client port. Thus ports on the remote host will act as the proxy for ports on the local machine.

The significant application of remote forwarding is that, suppose you have a local machine that lies inside an internal network connected to the Internet through a router